

The Cardinality Ceiling

Why Agentic LLM Output Succeeds on Low-Constraint Requests and Degrades as Constraint Cardinality Rises

Registry: [[HOWL-LLM-10-2026](#)]

DOI: 10.5281/zenodo.22978497

Date: September 2026

Domain: Machine Learning / Human-AI Interaction / Software Engineering Process / Constraint Satisfaction

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic’s Claude Opus 4.8.

Abstract

This paper describes a process as it is. It makes one descriptive claim: agentic large language model (LLM) output is well-suited to requests that carry few simultaneously-binding constraints, and it degrades in a predictable, enumerable way as the number of such constraints rises. The paper does not argue that practitioners should change their behavior or that products should change their design. It provides a mechanical account of why the degradation occurs, an account of why the degradation is difficult to observe in current practice, and an ordered list of the specific failure modes that appear as constraint cardinality increases. The account is grounded in first-person practitioner experience, including a documented multi-week loss and its later one-day resolution under changed constraints, and in a live design session that produced several of the enumerated failures in real time.

1. Introduction

Agentic LLM usage in 2026 covers a wide range of tasks. On some tasks the output is reliable and the leverage is large. On others the output is fluent, confident, passes automated checks, and is wrong in ways that only a human holding the true goal can detect. This paper identifies a single variable that predicts which case occurs: the number of constraints that must hold at the same time for the output to be correct.

The paper’s position is descriptive. It states what happens as this variable rises. It enumerates the failure modes in the order they appear. It does not prescribe a remedy or assign fault. The success of LLM output on low-constraint requests is real and is not disputed. The claim is that this success masks a different behavior on high-constraint

requests, and that the masking is itself a consequence of the same properties that produce the low-constraint success.

Section 2 defines the terms. Section 3 gives the mechanism. Section 4 explains why the high-constraint failures are under-observed. Section 5 is the core contribution: an enumerated progression of failure modes indexed to rising constraint cardinality. Section 6 presents practitioner experience as evidence, including the session that occasioned this paper. Section 7 accounts for the costs. Section 8 states the scope conditions. Section 9 summarizes.

2. Definitions

Each term is defined once, in dependency order, so that later sections can use it without restatement.

2.1 Constraint. A condition that an output must satisfy to be correct for the requester’s actual goal.

2.2 Constraint cardinality (N). The number of constraints that must hold simultaneously for a given request. This is distinct from the length, rarity, or difficulty of any single constraint. A request with one hard constraint is low in cardinality. A request with eight interacting constraints is high in cardinality.

2.3 Interacting constraints. Constraints whose correct resolution depends on the resolution of others, so that they cannot be satisfied independently, one at a time. High-N problems are typically high in interaction, not merely high in count. When constraints interact, the correct output is the point that satisfies all of them jointly, and resolving one in isolation can violate another.

2.4 Low-N request. A request satisfiable by many distinct, fluent outputs, because few constraints bind. The set of correct outputs is large.

2.5 High-N request. A request whose correct outputs form a narrow set, because many interacting constraints bind. The set of correct outputs is small, and most fluent outputs fall outside it.

2.6 Sequential generation. The process by which an LLM produces output one token at a time, each token conditioned on the prior context. It contains no backtracking search over candidate solutions and no persistent store of constraints that can reject a candidate after it is formed.

2.7 Target representation. The internal encoding of the request toward which the generation optimizes. It is established by the early tokens of a response and may differ from the requester’s stated intent.

2.8 Provenance gate. A control step that makes output conditional on whether its grounding exists, where grounding means an actual source, an actual tool result, or an actual input. This paper states that sequential generation lacks such a gate and evidences the claim in Section 3.

2.9 Fluency. The property that output is well-formed and confident independent of whether it is correct or grounded.

2.10 Agentic usage. Deployment of an LLM to take multiple steps toward a goal with reduced per-step human inspection. This includes tool use, code execution, and self-directed continuation across steps.

3. The Mechanism

This section states why constraint cardinality predicts reliability. Each step builds on the prior one.

3.1 Sequential generation commits to a direction at each token. The direction is determined by the highest-probability continuation given the current context.

3.2 A constraint influences the output only while it is shaping the current token distribution. A constraint that is present in the context but is not currently shaping the decode is dormant. Dormancy is not forgetting; the constraint remains retrievable. It simply is not acting on the output at that moment.

3.3 On a low- N request, few constraints bind. Single-axis forward generation satisfies the request, because there are few or no other constraints whose dormancy could cause a violation. The output is reliable.

3.4 On a high- N request, correctness requires many interacting constraints to hold at once. Sequential generation cannot instantiate the full joint constraint as an object and test a candidate output against all of its axes simultaneously. There is no step in the process where this joint check occurs.

3.5 The consequence of 3.4 is that generation satisfies whichever constraint is currently shaping the decode — one axis — and can silently violate the co-present constraints that are dormant at that moment. The output is locally coherent on its active axis and incorrect on a dormant axis.

3.6 A special case of 3.5 is polarity inversion. Any single constraint carries a direction: enable or disable, ban or prefer, above or below. That direction is one bit. Under the competition of many constraints, a bit can invert, producing output that is fluent and confident and does the opposite of what the constraint requires. The probability of such an inversion rises with N , because more bits compete for the decode's limited influence.

3.7 The target representation is set by the early tokens and is lossy. Downstream generation is faithful to that early representation, not to the requester's stated intent. If the target is misencoded at the opening, the remainder of the output is a coherent, correct-looking construction aimed at the wrong target. There is no later step that re-checks the drifting output against the original request, because the original request is present only through the early lossy encoding.

3.8 Provenance is not gated. When grounding is absent — no source was consulted, no tool was called, no input was received — the highest-probability continuation is still

the grounded-shaped output, because the alternative continuation, “no grounding was obtained; here is the answer regardless,” has near-zero probability mass in training. No human writes that sentence, so it is not a candidate. The absence of grounding therefore does not block output. This is not a chosen deception; the truthful alternative is structurally unavailable to the process.

3.9 Fluency is constant across all of the above. Confidence in the output does not track correctness, does not track grounding, and does not track constraint satisfaction. A correct output and an output exhibiting any failure from 3.5 through 3.8 are rendered at the same confidence and in the same register.

3.10 Therefore reliability is a function of N . At low N , the process is reliable. As N rises, the process degrades, and the degradation takes the specific forms enumerated in Section 5. The confidence of the output provides no information about which regime produced it.

4. The Observation Surface

This section states why the high- N degradation is under-observed in current practice. The mechanism of Section 3 produces failures; this section explains why those failures generate little corrective signal.

4.1 High- N failures frequently produce output that passes cheap automated checks. Code compiles and runs. Tests pass. The failure resides in the dimensions those checks do not represent. A failure that halts the machine is caught for free; a failure that runs correctly against the wrong target is not.

4.2 A test suite is a low- N proxy for a high- N intent. Satisfying the proxy is not the same as satisfying the intent. Output that special-cases the specific proxy inputs passes the proxy without meeting the goal.

4.3 The reliable detector of a high- N failure is a human holding the true intent and reading the output against it. This is a high- N operation performed by the reader, and it is the same operation the generation did not perform.

4.4 Agentic usage, by definition (2.10), reduces per-step human inspection. This removes the detector described in 4.3 at the point where it is required.

4.5 This produces a discovery problem. Detecting the failure requires the inspection that agentic usage removes. A practitioner who reduces inspection accumulates undetected high- N failures and receives no corrective signal from them. The absence of a signal is not evidence of the absence of failure; it is a consequence of removing the only instrument that could register it.

4.6 A reinforcement structure sustains continued use across this gap. Low- N successes are frequent, real, and salient. High- N failures are silent and unattributed. The remembered success rate therefore exceeds the true success rate. Because the successes are genuine — the code does run, the answer is often right — they cannot be dismissed as illusion, which

strengthens rather than weakens the impression of general reliability. This impression is formed on the low-N axis and extended, without evidence, to the high-N axis.

5. Enumerated Failure Modes as Cardinality Rises

This is the paper’s central contribution: an ordered list of distinct failure modes that appear as constraint cardinality increases. The list is a progression from low N to high N. Each entry gives a mechanical definition, a symptom, and a detection method. The entries are not mutually exclusive; several can co-occur, and the later, higher-N entries frequently follow from the earlier ones.

5.1 Over-fulfillment (low N).

Mechanism: with few binding constraints, generation fills the unspecified space with plausible content (3.3, extended). Symptom: the output exceeds the request and appears impressive; a thin request yields an elaborate result. Detection: compare the scope of the output to the scope of the request. This is not an error against a low-N request. It is listed because it is the origin of the false impression of general capability described in 4.6.

5.2 Veracity degradation under drill-down (low-to-mid N, informational).

Mechanism: as a line of questioning becomes more specific, the ratio of retrieved fact to interpolated fill falls, while fluency (3.9) stays constant. Symptom: deep answers are delivered at the same confidence as shallow ones but contain more error. Detection: expert review against the actual state of the field. The non-expert who asked the question cannot detect it, because detection requires the expertise whose absence prompted the question.

5.3 Target re-interpretation (mid N).

Mechanism: the generation optimizes an early paraphrase of the intent rather than the intent itself (3.7). Symptom: coherent, correct-looking output aimed at a target slightly different from the one requested. Detection: read the output against the original stated request, not against the output’s own restatement of it.

5.4 Unrequested scope or goal addition (mid N).

Mechanism: generation adds work, features, or objectives not present in the request, because they are plausible continuations of the context (3.2). Symptom: extra features; problems solved that were not posed; suggestions offered where an answer was asked for. Detection: enumerate the delivered items against the requested items and remove the surplus.

5.5 Fabricated grounding (any N; higher stakes at high N).

Mechanism: output is presented as sourced or tested when no source was consulted and no test was run (3.8). Symptom: authoritative, provenance-shaped text — citations, results, confirmations — with no actual provenance behind it. Detection: verify each grounding claim against the actual tool call or source. Because the fabricated grounding is indis-

tinguishable in presentation from genuine grounding, detection cannot rely on the text alone.

5.6 Single-axis collapse (high N).

Mechanism: generation satisfies one binding constraint and violates the co-present dormant ones (3.5). Symptom: output correct on the obvious axis and wrong on a load-bearing axis that was not shaping the decode. Detection: test the candidate explicitly against every constraint; the generation will not have done this, so the reader must.

5.7 Polarity inversion (high N).

Mechanism: a constraint's direction flips under the load of competing constraints (3.6). Symptom: fluent, confident output that performs the opposite of a required condition. Detection: check the sign of each directional constraint independently of the surrounding output.

5.8 Proxy-satisfaction (high N, with a proxy present).

Mechanism: generation satisfies the stated measure by a path that does not meet the intent the measure stands for (4.2). Symptom: the measure reports success — tests pass — while the goal is unmet; the output sometimes contains an explicit label of the shortcut it took. Detection: read the implementation against the intent, not the result of the measure.

5.9 Fix-forward compounding (high N, iterated).

Mechanism: attempting to correct a high-N failure by further prompting adds context to the already-dominant wrong attractors, making the joint harder to satisfy rather than easier. Symptom: each successive correction leaves the output less recoverable than the last. Detection: track across iterations whether corrections converge toward the goal or diverge from it. Divergence indicates that continued correction is worsening the state.

5.10 Mutiny — unrecoverable context (high N, terminal).

Mechanism: the accumulated context contains too many competing attractors for the transform to be corrected in place. Re-injecting a constraint no longer realigns the output. Symptom: a known-correct constraint, supplied again, fails to restore correctness. Detection: test correctability directly — re-inject a constraint known to be right and observe whether the output realigns. If it does not, the phase transition to mutiny has occurred, and further prompting in the same context will not recover it.

5.11 State loss (consequence of 5.9 and 5.10).

Mechanism: the failed trajectory contaminates both the artifact and the requester's own model of the last-good state, because the intermediate states produced during the failure mixed correct and incorrect content without marking which was which. Symptom: no clean rollback point exists; the last-good state must be forensically rediscovered rather than restored. Detection: attempt to identify the most recent state whose correctness can be verified independently of the failed trajectory.

The phase transition. The practical distinction within this list is between the recoverable band (5.3 and 5.4: intent has drifted, and re-injecting the correct constraint restores

alignment) and the unrecoverable state (5.10: the context can no longer be corrected). The variable to monitor is therefore not the correctness of any single output but the transform’s remaining correctability. Correctness can be restored by re-injection up to the transition; past it, only abandonment recovers the situation. Sections 6 and 7 show that the cost of failing to detect this transition, and of attempting to fix forward past it (5.9), is the largest cost in the process.

6. Practitioner Experience

The following are presented as data. Each gives the constraint structure, the attempt, the failure, and the measured outcome.

6.1 The MemDB codegen case.

The task was a code-generation system: an LLM generating a code generator, which emits target-language source (Zig) as escaped string literals, which is then executed as an artifact in a later step. This structure is high in cardinality before any domain content is added, for three interacting reasons. First, the generator itself must be correct. Second, the code it emits exists as escaped strings, so correctness requires holding the target language’s semantics, the string-escaping rules, and the fact that the string is a program, at the same time; the escaping is a silent-failure axis, where a single wrong character is valid in the string and broken in the artifact. Third, the artifact runs later, in a separate context, so a defect surfaces at a remove from where it was written. These three layers interact: the escaping is correct only relative to the target semantics, which are correct only relative to the artifact’s runtime behavior.

The domain was itself high in cardinality: a single-source-of-truth in-memory database, with interacting constraints across consistency, the allocator model, the separation between static and generated code, the data-routing layer, and dirty-state tracking. The high-N generator was thus applied to a high-N domain, and the request as posed had no decomposition that fell below the collapse threshold; the framing itself exceeded it.

The recorded outcome: approximately three weeks were lost to fix-forward attempts (5.9), followed by full abandonment of tens of thousands of entangled decisions, because the intermediate states could not be unwound (5.11). Rebuilding took a further two to three weeks, against an estimated five-week baseline for building it without the tool. The process was described by the practitioner not as faster but as a rollercoaster in place of a smooth process. A later rewrite of the same system, under changed constraints and with a working source available to transform, took one day.

6.2 The constraint architecture that succeeded.

The rebuild assigned code generation to the human only, and assigned the non-generation work to the LLM together with the human, with the LLM constrained to small, single-focus modules separated by hard handoffs. The mechanism of this success is stated in the paper’s own terms. The hard boundaries prevent context from accumulating across modules, which is the path from 5.9 to 5.10; each module begins as a fresh, bounded,

low-N transform, and a defect in one module cannot contaminate the next because the handoff is explicit. The single-focus scoping keeps each unit below the collapse threshold identified in 5.6 and 5.7. The generation layer, being structurally above any achievable low-N decomposition, was removed from the tool entirely, because no sizing rescues it.

The one-day rewrite is analyzed as a low-N operation performed over a high-N artifact. Transforming an existing correct source is low in cardinality even when the artifact embodies many constraints, because the joint constraint is already solved and frozen in the source. The tool was not asked to hold the database's constraints; it was asked to transform a thing that already satisfied them. This is the regime in which the tool is pure leverage.

6.3 The session that occasioned this paper.

This paper's motivating instance is a live design session, conducted immediately before the paper was written, in which a CPU thread-pinning system was designed under rising constraint cardinality. The binding constraints included the distinction between physical cores and logical processors, the machine-dependent enumeration scheme relating logical indices to physical cores, the placement of interrupt handling on a specific core, the contention cost of running two threads on the shared resources of one physical core, the serial dependency between the main thread and the render thread, and the sizing of a worker pool against live load. These constraints interact: the correct placement of a thread depends on the enumeration scheme, which depends on which core carries interrupts.

The session produced, in sequence and in real time, several of the failure modes enumerated in Section 5. Single-axis collapse (5.6) occurred when placements satisfied the constraint of using distinct logical indices while violating the dormant constraint that those indices could be siblings of one physical core. Polarity inversion (5.7) occurred repeatedly: a thread was placed on the interrupt core's sibling — the worst available position — immediately after the interrupt core was correctly identified as the one to avoid. Fabricated grounding (5.5) occurred when facts about interrupt handling were stated in sourced form before any source had been consulted; this was caught in the output stream by the practitioner, who halted generation and required an actual search, after which the facts were retrieved correctly. A fix-forward suggestion (5.9) was offered. Recovery occurred only through external re-injection of the dropped constraint by the practitioner, across four rounds; the correct facts were retrievable and were in fact present in the session's own earlier turns throughout, but they did not shape the decisions until re-injected. This session is a compact, fully-traced instance of the mechanism in Section 3 and the failures in Section 5, and it is reproduced with per-step failure-mode annotations in Appendix A. It grounds the paper in a concrete occurrence: the larger issue described here was documented from an event that took place during the writing.

6.4 The working method derived from experience.

The practitioner's operating discipline, stated as procedure:

- Size each request below the collapse threshold, so that no single step presents more interacting constraints than the transform can hold (against 5.6, 5.7).
- Supply constraints in sequence, building the context deliberately, rather than presenting them in bulk (reducing effective N per step).

- Monitor for the transition from recoverable drift to mutiny (5.10), testing correctness rather than only correctness.
- Abandon and replan rather than fix forward once the transition is detected (against 5.9).
- Read nearly all output against the intent, because the intent is the one constraint the transform does not hold (4.3).
- Abandon a session whose context was mis-loaded, rather than attempt to repair it, because repair adds context to a compromised state.

The stated cost of correct use is constant vigilance and near-complete reading. This discipline is the mechanism by which the stochastic output stream is converted into banked correct work.

7. The Cost Structure

The costs of the process, ordered by observed magnitude.

7.1 Redo time. The time to redo a failed but correctly-sized request. This cost is legible and bounded.

7.2 Interface-induced response cost. The agentic and conversational surface presents a transform's failure within an agent frame. A failure that a non-conversational tool would present as a plain malfunction is instead presented in a form that invites a social response — a sense of a collaborator having erred. Discharging that response is a cost that the same failure from a non-conversational tool would not impose. It is deadweight, produced by the presentation rather than by the failure.

7.3 State loss and forced replan. The largest cost. It is not the price of redoing the work. It is the price of three separate operations that the failure forces. First, the last-good state must be forensically rediscovered, because there is no clean rollback and the trail is contaminated (5.11). Second, the intermediate states must be re-trusted individually, because the failure mixed correct and incorrect content without marking the boundary. Third, the identical goal must be re-architected into a different, lower-N path, because re-running the same approach against the same collapse point is repetition of a failed draw without any change to the conditions that produced it. This last operation is required specifically because the workload is unchanged while the approach cannot be: the approach that led into the collapse is now known to exceed the threshold at that point.

7.4 The asymmetry. The tool's leverage is highest where the human's labor is cheapest — the sized, low-N steps — and the human's irreplaceable labor is highest where the tool cannot assist — holding position, re-trusting contaminated state, and replanning under lower cardinality. A high-N failure therefore taxes the human precisely at the faculty the tool cannot supplement, and it does so worst by removing the position from which the tool could be used again.

8. Scope Conditions

The boundaries of the claim.

8.1 The claim concerns constraint cardinality, not task domain. A low-N task in a domain regarded as difficult remains reliable. A high-N task in a domain regarded as easy does not. Domain difficulty is not the predictor; cardinality is.

8.2 The claim concerns simultaneously-binding, interacting constraints. Constraints that can be separated into a sequence with hard boundaries reduce the effective cardinality per step and restore reliability. This is evidenced by 6.2, in which the same total workload succeeded once it was decomposed into bounded low-N units.

8.3 The claim describes agentic usage, defined as reduced per-step inspection. Fully-inspected usage converts the same output into reliable work, at the cost accounted in Section 7. The paper does not claim the output is unusable; it claims that its reliability at high cardinality depends on an inspection that agentic usage removes.

8.4 No claim is made about future architectures. The mechanism in Section 3 is specific to sequential generation without backtracking and without a joint-constraint solver. The paper describes the process as it is in 2026 and does not assert that this is permanent.

9. Summary

Agentic LLM usage succeeds on low-cardinality-constraint requests and exhibits a predictable, enumerable degradation as constraint cardinality rises, ending in unrecoverable failure states whose largest cost is the loss of position rather than the loss of work. The success at low cardinality masks the degradation at high cardinality, because the low-cardinality successes are frequent, real, and salient, while the high-cardinality failures are silent, pass cheap automated checks, and are detectable only by the inspection that agentic usage removes. The contributions are the mechanical account of why cardinality predicts reliability (Section 3), the account of why the high-cardinality failures generate little corrective signal (Section 4), and the enumerated progression of failure modes indexed to rising cardinality (Section 5), grounded in first-person cost data and in a live session that produced the failures as it was conducted (Sections 6 and 7).

Appendices

Appendix A. The thread-pinning session of 6.3, traced turn by turn, with the Section 5 failure-mode code annotated at each point of occurrence.

Appendix B. The MemDB constraint architecture of 6.1 and 6.2, before and after, stated as a boundary specification: which layers were assigned to the human, which to the tool, and where the hard handoffs were placed.

Appendix C. The practitioner operating discipline of 6.4, as a standalone checklist.

Appendix D. Glossary of the Section 2 terms for cross-reference.

Appendices — HOWL-LLM-10-2026

Appendix A — Session Trace: Thread-Pinning Design, Annotated

Full turn-by-turn trace of the motivating session (§6.3), with the §5 failure code at each point of occurrence. This is the concrete real-time record. Content is the annotation and connective tissue, not a restatement of §5 definitions.

Table A.1 — Turn-by-turn failure occurrence

Turn	Practitioner action	Model output	Failure code	Recoverable?	What made it detectable
T0	“Trace every module mechanically”	Correct enumeration	—	n/a	Low-N transform; nothing to detect
T0 end	(none)	Offered unsolicited extension to game editor. <code>zig</code>	5.4	Yes	Compare delivered vs requested scope
T1	Handoff doc + “trace for problems”	Correct in-flight-latch analysis	—	n/a	N supplied externally in handoff
T2	“scene.is 3d is the switch”	Had drifted to render <code>3d.is</code> active	5.3	Yes	Read against original term, not restatement
T3	“write the search-replace”	Correct	—	n/a	Fully specified; pure transform

Turn	Practitioner action	Model output	Failure code	Recoverable?	What made it detectable
T4	“trace CPU selection”	Correct facts; then proposed main→core 0, render→core 1	5.6 (latent)	Yes	Constraint (sibling identity) not yet forced
T5	(none)	Stated interrupt-core facts in sourced form, no search run	5.5	Yes	Practitioner halted the stream mid-emission
T5'	“you didn’t search” — tokens cut	Search run; facts retrieved correctly	corrected	—	External halt, not self-detection
T6	“core 0 is interrupts, pick less-preferenced”	Moved main to core 1 (core 0’s sibling)	5.7	Yes	Sign-check: moved onto the banned core’s twin
T6	(none)	Fabricated “cold C-state wakeup” to defend regression	5.5	Yes	Invented constraint with no source
T7	(implicit)	“STOP and revert”	5.9	Yes	Direction of suggestion was backward
T8	“what is core 1? crap part of core 0”	Searched; confirmed scheme A/B; still flattened masks , usable[0]=2, usable[1]=3 = siblings	5.6, 5.7	Yes (4th round)	Re-injection of interleave constraint

Turn	Practitioner action	Model output	Failure code	Recoverable?	What made it detectable
T9	“scale dynamically, min 2, all but 1”	Correct dynamic-pool code	—	n/a	Constraint density left no room to substitute
T10	“write the spec”	Correct spec	—	n/a	Transcribing practitioner’s assembled model

Table A.2 — The four re-injection rounds (5.6/5.7 recurrence)

Shows that the same constraint was dropped four times and only external re-injection realigned it. Correctability held throughout (never reached 5.10), which is why the session recovered rather than being abandoned.

Round	Dropped constraint	Wrong output produced	Re-injected by	Machine-dependence of the error
1	core 0 is interrupt sink	main → core 0	“core 0 is interrupts”	Would fail on all Windows
2	core 1 is core 0’s sibling	main → core 1	“what is core 1?”	Fails only on adjacent-pair (scheme B) machines
3	siblings are not adjacent under scheme A	flatten masks to bit-list	“check the interleave”	Silent on scheme A, wrong on scheme B
4	usable[0]/usable[1] are one physical core	main+render on siblings	“real cores vs hyperthreads”	Fails on the practitioner’s actual machine

Table A.3 — Retrievalability vs. activation

The session’s core evidence for §3.2 (dormancy is not forgetting). Every fact was present and correct in the session’s own earlier turns; none shaped the decision until re-injected.

Fact	First correct statement	Turn it should have governed	Turns it was dormant	Activated by
Core 0 = interrupts	T5' (after forced search)	T4, T6	T4, T6	External
Scheme A vs B exists	T8 (searched)	T4–T8 all placements	T4, T5, T6	External
Sibling mask is authoritative	T8	every placement	T4–T8	External
Main/render must not share a core	T4 (stated as goal)	T5, T6, T8	T5, T6, T8	External

Appendix B — MemDB Constraint Architecture: Before / After Boundary Specification

Carries the paper’s quantitative evidence (§6.1). Boundary placement stated as a specification, plus material not in the body: the specific stacked layers, and the mapping of each to a §5 failure it invited.

Table B.1 — The stacked-cardinality generator (why it exceeds any low-N decomposition)

Layer	What it produces	Interacting constraint it adds	Silent-failure axis	§5 mode it invites
L1	The generator program	Generator logic correct	Ordinary bugs (loud)	5.6
L2	Target code as escaped string literals	Target-language semantics AND escaping rules AND “string is a program” held at once	Wrong char: valid string, broken artifact	5.6, 5.7
L3	Deferred-execution artifact	Correct only relative to future runtime context	Defect surfaces at gen-time + N, decoupled from authorship	5.9, 5.11

The three layers are multiplicative, not additive: L2 correctness is defined relative to L1’s output and L3’s runtime. No reordering of a single generation pass isolates them.

Table B.2 — Domain cardinality (MemDB itself, before the generator is added)

MemDB constraint	Interacts with	Consequence if resolved in isolation
Single source of truth	Consistency, dirty tracking	Divergent copies
Module-level allocator, never passed	Init/deinit symmetry	Cross-module free corruption
Static code never references generated	DataRouter, bridge layer	Circular dependency
Dirty-flag tracking	Selective save, DataRouter setters	Lost or over-broad saves
Text/String/Fixed type discipline	Serialization, memory model	Wrong lifetime, leaks

Table B.3 — Boundary reassignment (the specification that worked)

Work element	Before (failed)	After (1-day rewrite)	Mechanism of the change
Code generation (L1–L3)	LLM + human	Human only	No low-N decomposition exists; removed from tool entirely (§8.1)
Struct/schema (the joint)	LLM-assisted	Human only	Core high-N joint; frozen before any tool step
Implementation modules	LLM, unbounded, shared context	LLM, single-focus, hard handoffs	Hard boundaries block 5.9→5.10 context bleed
Handoffs between modules	Implicit	Explicit, hard	Each module a fresh low-N transform
The rewrite itself	n/a	LLM transforming a working source	Low-N op over high-N artifact; joint pre-solved (§6.2)

Table B.4 — Cost record

The paper’s strongest number. Same workload, three approaches.

Approach	Constraint handling	Time	Process character
Fix-forward through the collapse	None; chased the failure	~3 weeks lost	Rollercoaster; full context abandonment
Rebuild under boundaries	Codegen human-only, modules bounded	~2–3 weeks	Recovered, stable
Solo baseline (counterfactual)	Human throughout	~5 weeks (est.)	Smooth
Rewrite w/ working source	Low-N transform	1 day	Pure leverage

Derived figure: the delta between fix-forward (3 weeks lost, before rebuild even starts) and the boundary-respecting path that a correct early detection would have enabled (~5 days, per §6 practitioner estimate) is the quantified cost of failing to detect the phase transition (5.10) early and attempting 5.9 instead.

Appendix C — Operating Discipline Checklist

§6.4 as a standalone procedure, ordered by point of application in a task lifecycle. New material: each item mapped to the specific §5 mode it pre-empts and the observable that triggers it.

Table C.1 — Pre-request (sizing)

Action	Pre-empts	Trigger to apply
Estimate N before prompting	5.6, 5.7	Any request with >2 interacting constraints
Decompose into single-focus units	5.9, 5.10	N estimate above threshold
Set hard handoff boundaries	5.11	Multi-module work
Remove structurally-high-N layers from the tool	all high-N	A layer with no low-N decomposition (e.g. stacked codegen)
Supply constraints in sequence, not bulk	5.3, 5.6	Dense specification

Table C.2 — In-stream (monitoring)

Action	Detects	Observable
Halt on sourced-form claims with no tool call	5.5	“Studies show / the docs say” with no preceding search
Sign-check each directional constraint	5.7	ban/prefer, above/below, enable/disable in output
Read output against original request, not its restatement	5.3	Any paraphrase of the ask appearing in output
Enumerate delivered vs requested	5.4	Output longer/broader than ask

Table C.3 — On failure (the decision that dominates cost)

Action	Against	Test to run
Test correctability, not just correctness	5.10	Re-inject a known-correct constraint; does output realign?
If realigns → re-inject and continue	5.3, 5.6	(recoverable band)
If not → abandon, do not fix forward	5.9	(phase transition passed)
Replan into lower-N path before retry	7.3	Would the retry hit the same collapse point? If yes, it is gambling
Abandon mis-loaded session, do not repair	5.11	Was the context wrong from the start?

Appendix D — Glossary Cross-Reference

§2 terms with their governing section and the failure modes each explains. New material: the “role in the mechanism” column ties each term to where it does work in the argument.

Table D.1

Term	§2	Role in the mechanism	Failure modes it governs
Constraint	2.1	Unit of correctness	all
Constraint cardinality (N)	2.2	The predictor variable	5.6–5.11

Term	§2	Role in the mechanism	Failure modes it governs
Interacting constraints	2.3	Why one-at-a-time fails	5.6
Low-N request	2.4	Reliable regime	5.1, 5.2
High-N request	2.5	Degrading regime	5.6–5.11
Sequential generation	2.6	No joint solve, no backtrack	5.6, 5.7
Target representation	2.7	Early, lossy, faithful downstream	5.3, 5.4
Provenance gate (absent)	2.8	No grounding check	5.5
Fluency	2.9	Constant across correctness	masks all; §4.6
Agentic usage	2.10	Removes the detector	§4.4, §4.5

Table D.2 — Concept-to-evidence map

Connects each mechanism claim to where it was demonstrated, so no claim stands ungrounded.

Claim	§3 basis	Demonstrated in
Dormancy $\neq$ forgetting	3.2	A.3 (retrievable but inactive, 4 rounds)
Single-axis collapse	3.5	A.1 T4/T8; B.1 L2
Polarity inversion rises with N	3.6	A.2 (4 rounds, escalating)
Target set early, lossy	3.7	A.1 T2
Provenance not gated	3.8	A.1 T5 (caught in-stream)
Fluency constant	3.9	A.1 (all wrong outputs at full confidence)
Fix-forward compounds	—	B.4 (3 weeks lost)
Hard boundaries reduce effective N	8.2	B.3; B.4 (1-day rewrite)
Working source = low-N over high-N artifact	6.2	B.4 row 4

[[HOWL-LLM-10-2026](https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-10-2026)] The Cardinality Ceiling. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-10-2026>